Materia: Lógica de Programación

S1. Tarea. Modularización del código

Sebastian Zapata Alvarez

Profesor: Edwin Ferney Barragán Muñoz

Institución Universitaria Pascual Bravo

Medellín

2026

REPORTE DE REFACTORIZACION

Sistema de Inventario GadgetXpress en C++

1. Introducción

El presente trabajo consiste en la refactorización de un programa desarrollado en lenguaje C++ para la gestión de inventario de la empresa GadgetXpress.

El objetivo principal fue mejorar la estructura, organización y legibilidad del código fuente, aplicando el principio de responsabilidad única. Esto permite que cada función realice una tarea específica, facilitando el mantenimiento, la escalabilidad y futuras mejoras del sistema.

La refactorización realizada transforma un programa funcional en un código más profesional, modular y facil de entender.

2. Objetivo de la refactorización

La refactorización tuvo como finalidad:

- Mejorar la organización del programa.
- Dividir el código en funciones específicas.
- Evitar repetición de código.
- Simplificar la función principal main.
- Facilitar futuras modificaciones y mantenimiento.
- Mejorar la legibilidad y comprensión del sistema.

3. Proceso de refactorización

3.1 División del programa en funciones

El programa fue dividido en funciones independientes, donde cada una cumple una responsabilidad específica.

Se implementaron las siguientes funciones:

Funcion	Responsabilidad
mostrarMenu()	Mostrar las opciones del sistema
agregarProducto()	Registrar productos

listarProductos()	Mostrar inventario
actualizarCantidad()	Modificar cantidades
generarReporteBajoInventario()	Mostrar productos con bajo stock
buscarProducto()	Buscar información de productos
buscarProductoPorCodigo()	Función auxiliar reutilizable
validarCantidad()	Validar cantidades
validarPrecio()	Validar precios

3.2 Creacion de funciones auxiliares

Durante la refactorización se identificó que la búsqueda de productos por código se repetía en diferentes partes del programa.

Por esta razón, se creó la función:

int buscarProductoPorCodigo(...)

Esta función permite reutilizar la lógica de búsqueda en varias funcionalidades del sistema, evitando duplicación de código y mejorando el mantenimiento.

También se implementaron funciones auxiliares para validaciones y limpieza del buffer de entrada.

4. Antes y después de la funcion main

4.1 Antes de la refactorización

Inicialmente, la función main contenía demasiadas responsabilidades:

- Mostrar menu
- Validar entradas
- Ejecutar operaciones
- Controlar el flujo completo
- Manejar mensajes del sistema

Esto hacía que el código fuera más largo, difícil de leer y complicado de mantener.

Ejemplo simplificado del main original

```
do {
  cout << "1. Registrar productos";
  cout << "2. Mostrar productos";
  cout << "3. Actualizar cantidad";
```

```
  cin >> opcion;

  switch(opcion) {
     case 1:
        registrarProductos(...);
        break;

     case 2:
        mostrarProductos(...);
        break;
  }

} while(opcion != 0);
```

4.2 Despues de la refactorizacion

Luego de la refactorización, la función main quedó mucho más limpia y organizada.

Ahora su única responsabilidad es:

- Mantener el ciclo principal.
- Llamar al menu.
- Ejecutar funciones según la opción elegida.

Main refactorizado

```
do {
  mostrarMenu();
  cin >> opcion;

  switch(opcion) {

     case 1:
        agregarProducto(productos, totalProductos);
        break;

     case 2:
        listarProductos(productos, totalProductos);
        break;

     case 3:
        actualizarCantidad(productos, totalProductos);
        break;
  }

} while(opcion != 0);
```

5. Beneficios de la nueva estructura

La nueva estructura del programa ofrece varias ventajas importantes:

Modularidad

Cada función realiza una tarea específica y puede modificarse de manera independiente.

Reutilizacion de codigo

Funciones auxiliares como buscarProductoPorCodigo() evitan repetir lógica innecesariamente.

Mejor mantenimiento

El código es más fácil de corregir, actualizar y mejorar.

Escalabilidad

La estructura modular facilita agregar futuras funcionalidades, como simulación de ventas o eliminación de productos.

Legibilidad

El programa es más fácil de entender para otros desarrolladores.

6. Conclusiones

La refactorización realizada permitió transformar un programa funcional en una solución más profesional y organizada.

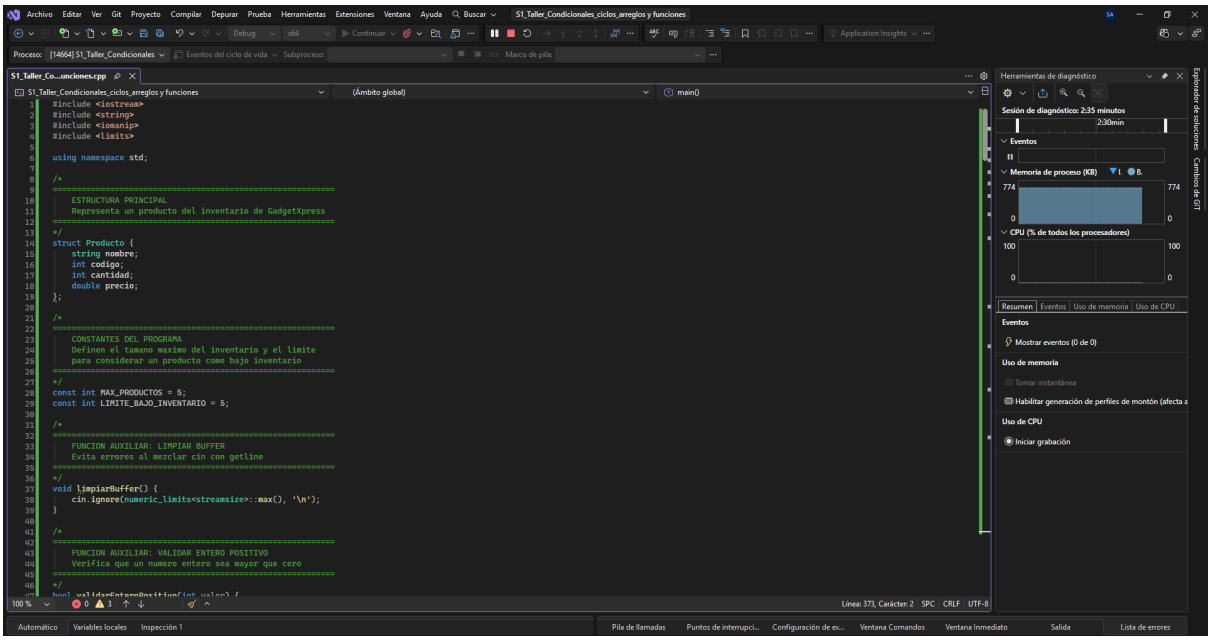
La división del sistema en funciones específicas mejoró significativamente la claridad del código y redujo la complejidad de la función principal.

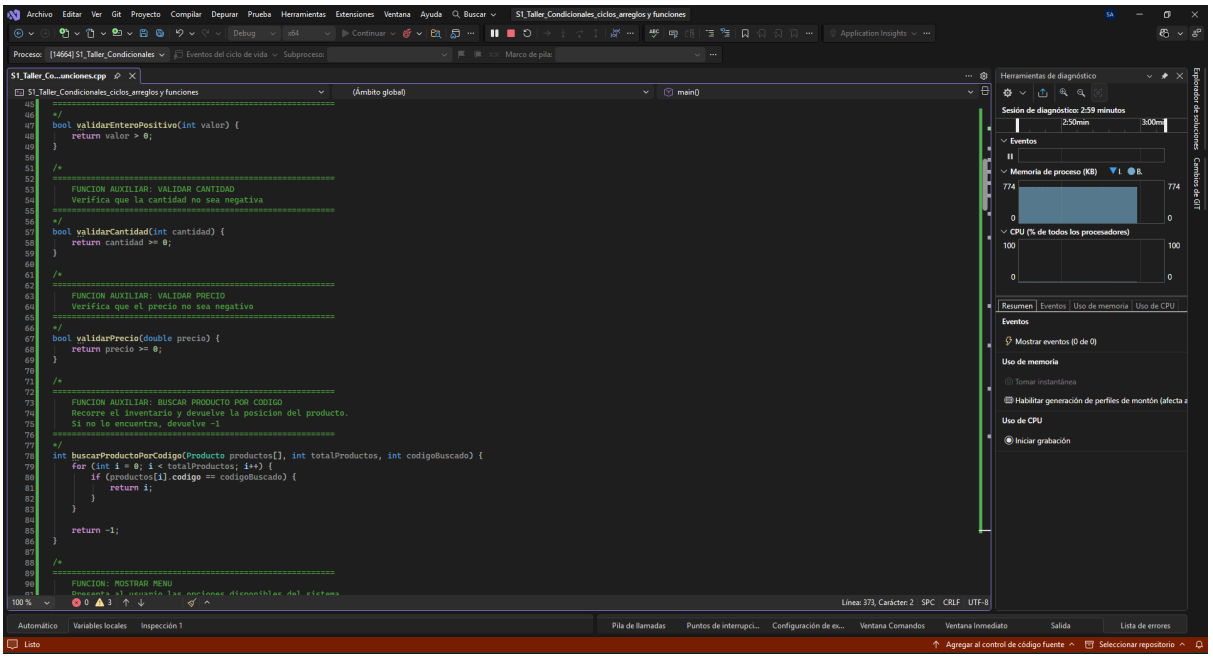
Además, el uso de funciones auxiliares facilita la reutilización de lógica y prepara el sistema para futuras ampliaciones.

Finalmente, esta experiencia permitió aplicar buenas prácticas de programación modular en C + +, fortaleciendo habilidades relacionadas con el desarrollo de software mantenible y escalable.

7. Evidencias de funcionamiento

En esta sección se anexan capturas de pantalla que demuestran el correcto funcionamiento del sistema:





```cpp
    FUNCION: MOSTRAR MENU
    Presenta al usuario las opciones disponibles del sistema
==========================================================
*/
void mostrarMenu() {
    cout << "\n=====================================\n";
    cout << "          SISTEMA GADGETXPRESS\n";
    cout << "=====================================\n";
    cout << "1. Agregar producto\n";
    cout << "2. Listar productos\n";
    cout << "3. Actualizar cantidad\n";
    cout << "4. Generar reporte de bajo inventario\n";
    cout << "5. Buscar producto\n";
    cout << "0. Salir\n";
    cout << "Seleccione una opcion: ";
}

/*
==========================================================
    FUNCION: AGREGAR PRODUCTO
    Permite registrar un nuevo producto en el inventario,
    validando que el codigo no se repita y que los datos
    sean correctos
==========================================================
*/
void agregarProducto(Producto productos[], int& totalProductos) {
    if (totalProductos >= MAX_PRODUCTOS) {
        cout << "\nNo se pueden agregar mas productos. Inventario lleno.\n";
        return;
    }

    Producto nuevoProducto;

    cout << "\n========== AGREGAR PRODUCTO ==========\n";

    limpiarBuffer();

    cout << "Nombre del producto: ";
    getline(cin, nuevoProducto.nombre);

    cout << "Codigo: ";
    cin >> nuevoProducto.codigo;

    while (!validarEnteroPositivo(nuevoProducto.codigo) ||
        buscarProductoPorCodigo(productos, totalProductos, nuevoProducto.codigo) != -1) {

        if (!validarEnteroPositivo(nuevoProducto.codigo)) {
            cout << "Error. El codigo debe ser mayor que cero: ";
        }
        else {
            cout << "Error. El codigo ya existe. Ingrese otro codigo: ";
        }

        cin >> nuevoProducto.codigo;
    }

    cout << "Cantidad: ";
    cin >> nuevoProducto.cantidad;

    while (!validarCantidad(nuevoProducto.cantidad)) {
        cout << "Error. La cantidad no puede ser negativa: ";
        cin >> nuevoProducto.cantidad;
    }

    cout << "Precio: ";
    cin >> nuevoProducto.precio;

    while (!validarPrecio(nuevoProducto.precio)) {
        cout << "Error. El precio no puede ser negativo: ";
        cin >> nuevoProducto.precio;
    }

    productos[totalProductos] = nuevoProducto;
    totalProductos++;

    cout << "\nProducto agregado correctamente.\n";
}

/*
==========================================================
    FUNCION: LISTAR PRODUCTOS
    Muestra todos los productos registrados en una tabla
    organizada, incluyendo el valor total en inventario
==========================================================
*/
void listarProductos(Producto productos[], int totalProductos) {
    if (totalProductos == 0) {
        cout << "\nNo hay productos registrados.\n";
        return;
    }

    cout << "\n========================= LISTA DE PRODUCTOS =========================\n";
```

```cpp
    cout << "\n======================== LISTA DE PRODUCTOS ========================\n";

    cout << left << setw(25) << "Nombre"
        << setw(10) << "Codigo"
        << setw(10) << "Cantidad"
        << setw(12) << "Precio"
        << setw(15) << "Valor total" << endl;

    cout << "-----------------------------------------------------------------------\n";

    for (int i = 0; i < totalProductos; i++) {
        double valorTotal = productos[i].cantidad * productos[i].precio;

        cout << left << setw(25) << productos[i].nombre
            << setw(10) << productos[i].codigo
            << setw(10) << productos[i].cantidad
            << "$" << setw(11) << fixed << setprecision(2) << productos[i].precio
            << "$" << setw(14) << fixed << setprecision(2) << valorTotal
            << endl;
    }
}

/*
=========================================================
    FUNCION: ACTUALIZAR CANTIDAD
    Busca un producto por codigo y permite modificar
    su cantidad disponible en inventario
=========================================================
*/
void actualizarCantidad(Producto productos[], int totalProductos) {
    if (totalProductos == 0) {
        cout << "\nNo hay productos registrados.\n";
        return;
    }

    int codigoBuscado;
    cout << "\nIngrese el codigo del producto a actualizar: ";
    cin >> codigoBuscado;

    int posicion = buscarProductoPorCodigo(productos, totalProductos, codigoBuscado);

    if (posicion == -1) {
        cout << "\nProducto no encontrado.\n";
        return;
    }
```

```cpp
    }

    cout << "\nProducto encontrado: " << productos[posicion].nombre << endl;
    cout << "Cantidad actual: " << productos[posicion].cantidad << endl;

    cout << "Nueva cantidad: ";
    cin >> productos[posicion].cantidad;

    while (!validarCantidad(productos[posicion].cantidad)) {
        cout << "Error. La cantidad no puede ser negativa: ";
        cin >> productos[posicion].cantidad;
    }

    cout << "\nCantidad actualizada correctamente.\n";
}

/*
=========================================================
    FUNCION: GENERAR REPORTE DE BAJO INVENTARIO
    Muestra los productos cuya cantidad sea menor al limite
    establecido para reabastecimiento
=========================================================
*/
void generarReporteBajoInventario(Producto productos[], int totalProductos) {
    if (totalProductos == 0) {
        cout << "\nNo hay productos registrados.\n";
        return;
    }

    bool hayProductosBajos = false;

    cout << "\n============= REPORTE DE BAJO INVENTARIO =============\n";

    cout << left << setw(25) << "Nombre"
        << setw(10) << "Codigo"
        << setw(10) << "Cantidad"
        << setw(12) << "Precio"
        << "Estado" << endl;

    cout << "-----------------------------------------------------------------\n";

    for (int i = 0; i < totalProductos; i++) {
        if (productos[i].cantidad < LIMITE_BAJO_INVENTARIO) {
            cout << left << setw(25) << productos[i].nombre
                << setw(10) << productos[i].codigo
                << setw(10) << productos[i].cantidad
```

```cpp
                << setw(10) << productos[i].cantidad
                << "$" << setw(11) << fixed << setprecision(2) << productos[i].precio
                << "Reabastecer urgente" << endl;

            hayProductosBajos = true;
        }
    }

    if (!hayProductosBajos) {
        cout << "No hay productos con bajo inventario.\n";
    }
}

/*
==========================================================
    FUNCION: BUSCAR PRODUCTO
    Permite consultar toda la informacion de un producto
    especifico a partir de su codigo
==========================================================
*/
void buscarProducto(Producto productos[], int totalProductos) {
    if (totalProductos == 0) {
        cout << "\nNo hay productos registrados.\n";
        return;
    }

    int codigoBuscado;
    cout << "\nIngrese el codigo del producto a buscar: ";
    cin >> codigoBuscado;

    int posicion = buscarProductoPorCodigo(productos, totalProductos, codigoBuscado);

    if (posicion == -1) {
        cout << "\nProducto no encontrado.\n";
        return;
    }

    double valorTotal = productos[posicion].cantidad * productos[posicion].precio;

    cout << "\nProducto encontrado:\n";
    cout << "Nombre: " << productos[posicion].nombre << endl;
    cout << "Codigo: " << productos[posicion].codigo << endl;
    cout << "Cantidad: " << productos[posicion].cantidad << endl;
    cout << "Precio: $" << fixed << setprecision(2) << productos[posicion].precio << endl;
    cout << "Valor total en inventario: $" << fixed << setprecision(2) << valorTotal << endl;
}
```

```cpp
}

/*
==========================================================
    FUNCION PRINCIPAL
    Controla el ciclo principal del sistema y llama las
    funciones correspondientes segun la opcion seleccionada
==========================================================
*/
int main() {
    Producto productos[MAX_PRODUCTOS];
    int totalProductos = 0;
    int opcion;

    do {
        mostrarMenu();
        cin >> opcion;

        if (cin.fail()) {
            cin.clear();
            limpiarBuffer();
            cout << "\nEntrada invalida. Debe ingresar un numero.\n";
            continue;
        }

        switch (opcion) {
        case 1:
            agregarProducto(productos, totalProductos);
            break;

        case 2:
            listarProductos(productos, totalProductos);
            break;

        case 3:
            actualizarCantidad(productos, totalProductos);
            break;

        case 4:
            generarReporteBajoInventario(productos, totalProductos);
            break;

        case 5:
            buscarProducto(productos, totalProductos);
            break;

        case 6:
```

```cpp
        if (cin.fail()) {
            cin.clear();
            limpiarBuffer();
            cout << "\nEntrada invalida. Debe ingresar un numero.\n";
            continue;
        }

        switch (opcion) {
        case 1:
            agregarProducto(productos, totalProductos);
            break;

        case 2:
            listarProductos(productos, totalProductos);
            break;

        case 3:
            actualizarCantidad(productos, totalProductos);
            break;

        case 4:
            generarReporteBajoInventario(productos, totalProductos);
            break;

        case 5:
            buscarProducto(productos, totalProductos);
            break;

        case 0:
            cout << "\nSaliendo del sistema GadgetXpress...\n";
            break;

        default:
            cout << "\nOpcion invalida. Intente nuevamente.\n";
            break;
        }

    } while (opcion != 0);

    return 0;
}
```

```
C:\Users\tedya\OneDrive\Escr

====================================
        SISTEMA GADGETXPRESS
====================================
1. Agregar producto
2. Listar productos
3. Actualizar cantidad
4. Generar reporte de bajo inventario
5. Buscar producto
0. Salir
Seleccione una opcion: 1

========== AGREGAR PRODUCTO ==========
Nombre del producto: Teclado
Codigo: 123
Cantidad: 30
Precio: 20000

Producto agregado correctamente.

====================================
        SISTEMA GADGETXPRESS
====================================
1. Agregar producto
2. Listar productos
3. Actualizar cantidad
4. Generar reporte de bajo inventario
5. Buscar producto
0. Salir
Seleccione una opcion: 1

========== AGREGAR PRODUCTO ==========
Nombre del producto: Monitor
Codigo: 124
Cantidad: 10
Precio: 120000

Producto agregado correctamente.
```

```
C:\Users\tedya\OneDrive\Escr

====================================
        SISTEMA GADGETXPRESS
====================================
1. Agregar producto
2. Listar productos
3. Actualizar cantidad
4. Generar reporte de bajo inventario
5. Buscar producto
0. Salir
Seleccione una opcion: 1

========== AGREGAR PRODUCTO ==========
Nombre del producto: Control Xbox
Codigo: 125
Cantidad: 30
Precio: 60000

Producto agregado correctamente.

====================================
        SISTEMA GADGETXPRESS
====================================
1. Agregar producto
2. Listar productos
3. Actualizar cantidad
4. Generar reporte de bajo inventario
5. Buscar producto
0. Salir
Seleccione una opcion: 1

========== AGREGAR PRODUCTO ==========
Nombre del producto: Diademas
Codigo: 126
Cantidad: 12
Precio: 30000

Producto agregado correctamente.
```

```
C:\Users\tedya\OneDrive\Esci

=========== AGREGAR PRODUCTO ===========
Nombre del producto: Diademas
Codigo: 126
Cantidad: 12
Precio: 30000

Producto agregado correctamente.

====================================
        SISTEMA GADGETXPRESS
====================================
1. Agregar producto
2. Listar productos
3. Actualizar cantidad
4. Generar reporte de bajo inventario
5. Buscar producto
0. Salir
Seleccione una opcion: 1

=========== AGREGAR PRODUCTO ===========
Nombre del producto: Parlantes
Codigo: 127
Cantidad: 12
Precio: 40000

Producto agregado correctamente.

====================================
        SISTEMA GADGETXPRESS
====================================
1. Agregar producto
2. Listar productos
3. Actualizar cantidad
4. Generar reporte de bajo inventario
5. Buscar producto
0. Salir
Seleccione una opcion: 1

No se pueden agregar mas productos. Inventario lleno.
```

```
C:\Users\tedya\OneDrive\Esci

No se pueden agregar mas productos. Inventario lleno.

====================================
        SISTEMA GADGETXPRESS
====================================
1. Agregar producto
2. Listar productos
3. Actualizar cantidad
4. Generar reporte de bajo inventario
5. Buscar producto
0. Salir
Seleccione una opcion: 2

======================== LISTA DE PRODUCTOS ========================
Nombre                    Codigo    Cantidad  Precio      Valor total
-------------------------------------------------------------------------
Teclado                   123       30        $20000.00   $600000.00
Monitor                   124       10        $120000.00  $1200000.00
Control Xbox              125       30        $60000.00   $1800000.00
Diademas                  126       12        $30000.00   $360000.00
Parlantes                 127       12        $40000.00   $480000.00

====================================
        SISTEMA GADGETXPRESS
====================================
1. Agregar producto
2. Listar productos
3. Actualizar cantidad
4. Generar reporte de bajo inventario
5. Buscar producto
0. Salir
Seleccione una opcion: 3

Ingrese el codigo del producto a actualizar: 123

Producto encontrado: Teclado
Cantidad actual: 30
Nueva cantidad: 10

Cantidad actualizada correctamente.
```

```
C:\Users\tedya\OneDrive\Esci

=============== REPORTE DE BAJO INVENTARIO ===============
Nombre                    Codigo    Cantidad  Precio      Estado
-------------------------------------------------------------
No hay productos con bajo inventario.

====================================
        SISTEMA GADGETXPRESS
====================================
1. Agregar producto
2. Listar productos
3. Actualizar cantidad
4. Generar reporte de bajo inventario
5. Buscar producto
0. Salir
Seleccione una opcion: 3

Ingrese el codigo del producto a actualizar: 123

Producto encontrado: Teclado
Cantidad actual: 10
Nueva cantidad: 0

Cantidad actualizada correctamente.

====================================
        SISTEMA GADGETXPRESS
====================================
1. Agregar producto
2. Listar productos
3. Actualizar cantidad
4. Generar reporte de bajo inventario
5. Buscar producto
0. Salir
Seleccione una opcion: 4

=============== REPORTE DE BAJO INVENTARIO ===============
Nombre                    Codigo    Cantidad  Precio      Estado
-------------------------------------------------------------
Teclado                   123       0         $20000.00   Reabastecer urgente
```

```
=====================================
        SISTEMA GADGETXPRESS
=====================================
1. Agregar producto
2. Listar productos
3. Actualizar cantidad
4. Generar reporte de bajo inventario
5. Buscar producto
0. Salir
Seleccione una opcion: 5

Ingrese el codigo del producto a buscar: 123

Producto encontrado:
Nombre: Teclado
Codigo: 123
Cantidad: 0
Precio: $20000.00
Valor total en inventario: $0.00

=====================================
        SISTEMA GADGETXPRESS
=====================================
1. Agregar producto
2. Listar productos
3. Actualizar cantidad
4. Generar reporte de bajo inventario
5. Buscar producto
0. Salir
Seleccione una opcion: |
```

```
=====================================
        SISTEMA GADGETXPRESS
=====================================
1. Agregar producto
2. Listar productos
3. Actualizar cantidad
4. Generar reporte de bajo inventario
5. Buscar producto
0. Salir
Seleccione una opcion: 0

Saliendo del sistema GadgetXpress...

C:\Users\tedya\OneDrive\Escritorio\C++\S1_Taller_Condicionales_ciclos_arreglos y funciones\x64\Debug\S1_Taller_Condicionales_ciclos_arreglos y funciones.exe (proceso 14664) se cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramientas ->Opciones ->Depuración ->Cerrar la consola automáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . .|
```